Optimisation hivernale du déneigement à Montréal

Akshay Ramael, Bastien Desert, Enzo Revel, Lucas Ly-Ba, Médéric Lavirotte

Résumé du projet et périmètre considéré

Chaque hiver, la ville de Montréal est confrontée à des épisodes neigeux fréquents pouvant aller jusqu'à 15 cm de neige. Notre travail porte sur les opérations de **déblaiement** (neige entre 2,5 et 15 cm) et cherche à optimiser la logistique des opérations en deux phases :

— phase 1 : survol des routes par des drones de reconnaissance ;
— phase 2 : planification des trajets de déneigement.

Les données proviennent d'un fichier GeoJSON contenant la description géométrique du réseau routier de Montréal sous forme de segments (LineString). Avant traitement, seules les routes accessibles aux véhicules (routes carrossables) sont conservées. Les données sont ensuite transformées en graphe, où chaque nœud représente un point du réseau et chaque arête un tronçon de route, permettant l'application des algorithmes de parcours.

1 Hypothèses et choix de modélisation

Nos principales hypothèses sont :

— La vitesse des drones est fixée à 40 km/h.
— Les drones et les véhicules ont une autonomie illimitée (pas de contrainte de batterie ou de recharge modélisée).
— Lorsqu'il y a plusieurs drones ou plusieurs véhicules, ceux-ci sont envoyés simultanément et parcourent chacun leur zone en parallèle.

Modélisation des drones

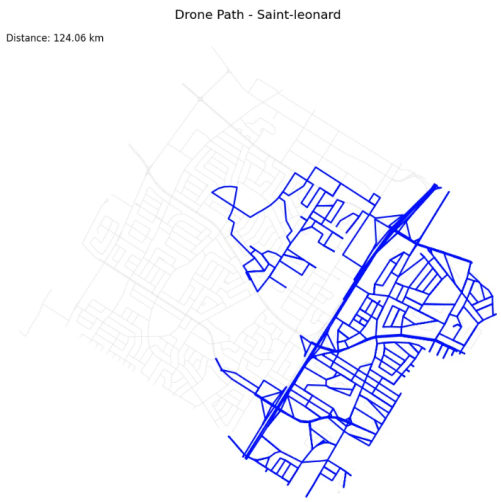
Nous avons conçu un programme permettant de simuler et visualiser le trajet d'un ou plusieurs drones de reconnaissance sur le réseau routier de Montréal, à partir des données issues d'un fichier GeoJSON. Le programme génère une animation `.mp4` représentant le parcours du drone et calcule le cout ainsi que le temps estimé de mission.

Le fichier GeoJSON est d'abord chargé via `GeoPandas`, et seuls les objets de type `LineString` (segments de voirie) sont conservés. Chaque segment est converti en arêtes d'un graphe non orienté `networkx.Graph`, où chaque sommet correspond aux coordonnées géographiques (longitude, latitude) des extrémités du segment.

Dans un premier temps, on extrait la plus grande composante connexe du graphe afin de ne conserver que la zone entièrement accessible pour le drone. Afin d'optimiser le trajet de surveillance, on applique un algorithme basé sur le problème du postier chinois (Chinese Postman Problem). Celui-ci permet de générer un circuit qui couvre l'intégralité des segments du graphe en minimisant les répétitions de trajets. Si le graphe n'est pas eulérien, des arêtes sont ajoutées à l'aide de la fonction `eulerize()` pour équilibrer les degrés des sommets.

Une fois le circuit obtenu, celui-ci est converti en séquence de segments géométriques que le drone suivra. La distance totale est calculée en sommant les longueurs individuelles de chaque segment, et le temps de parcours est déduit en considérant une vitesse constante de 40 km/h pour le drone.

L'animation est ensuite générée avec `matplotlib.animation`. Le trajet du drone est progressivement tracé à l'écran, permettant de visualiser son avancée sur l'ensemble du réseau routier. Le fichier `.mp4` est ensuite exporté à l'emplacement spécifié.



Enfin, le programme retourne le coût d'exploitation ainsi que le temps total nécessaire pour réaliser le parcours complet de Montréal.

Modélisation des déneigeuses

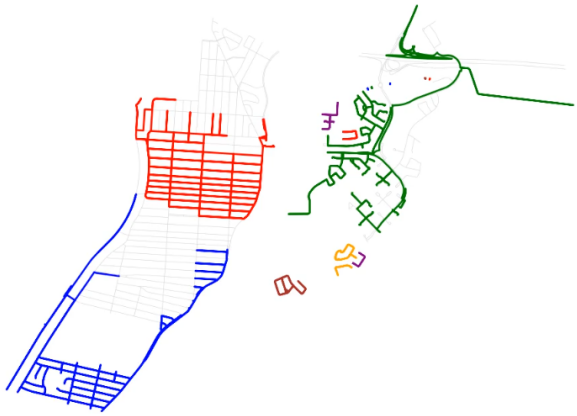
Pour ce qui est des déneigeuses, nous avons développé un programme qui simule et visualise le trajet de plusieurs déneigeuses sur un réseau routier défini dans un fichier GeoJSON. Il génère en sortie une animation `.mp4` représentant les déplacements, tout en calculant la distance totale et maximale parcourue.

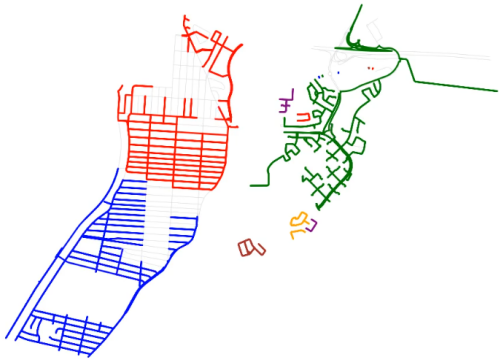
Dans un premier temps, le fichier GeoJSON est chargé via `GeoPandas`, et seuls les objets géométriques de type `LineString` sont conservés (correspondant aux routes). Chaque ligne est transformée en arêtes dans un graphe orienté `networkx.DiGraph`, où chaque sommet représente un point géographique (latitude, longitude). Si la route n'est pas à sens unique, une arête dans chaque direction est ajoutée.

Le graphe complet est ensuite décomposé en composantes fortement connexes pour s'assurer que chaque sous-graphe est navigable sans impasse. La fonction `divide_graph_for_deneigeuses` divise la plus grande composante en un nombre de zones égales au nombre de déneigeuses. Cette répartition repose sur la distance des sommets à un nœud central (ayant le plus de connexions), afin de créer des zones de taille similaire. Ensuite, pour chaque sous-graphe, on vérifie qu'il est bien fortement connexe. Si ce n'est pas le cas, des arêtes supplémentaires sont ajoutées pour relier les composantes disjointes.

Ensuite, pour chaque sous-graphe, on calcule un circuit eulérien à l'aide de la fonction `calculate_circuit`. Cette opération garantit que la déneigeuse pourra parcourir toutes les routes sans répétition inutile. Si le graphe n'est pas eulérien (c'est-à-dire si le degré entrant et sortant de certains nœuds diffère), des arêtes sont ajoutées pour équilibrer les chemins.

Une animation est ensuite générée avec `matplotlib.animation`. Chaque déneigeuse est représentée par une couleur différente, et ses déplacements sont mis à jour au fur et à mesure selon le circuit calculé. Le fichier `.mp4` est sauvegardé à l'emplacement spécifié.





Enfin, les distances parcourues par chaque déneigeuse sont additionnées pour donner une idée de la charge totale (et maximale) de travail. La fonction retourne ces deux métriques.

2 Scénarios de simulation et indicateurs

Résultats pour les drones de reconnaissance

Nous avons simulé trois scénarios de survol des routes par des drones de reconnaissance afin d'évaluer les différentes stratégies de couverture du territoire.

Scénario 1 : 19 drones, 1 par quartier administratif

Chaque drone traite un arrondissement indépendant. Cette répartition est simple à organiser sur le plan opérationnel, chaque quartier disposant de son propre drone dédié.

Scénario 2 : 10 drones découpage algorithmique

Le territoire de Montréal est automatiquement divisé en 10 zones équilibrées à l'aide d'un algorithme de partitionnement du graphe. Chaque drone survole sa propre zone de manière simultanée, ce qui permet de répartir équitablement la charge de travail entre les drones, de limiter les temps de parcours individuels et d'optimiser le temps total d'intervention. Ce scénario est également moins coûteux que le scénario 1 car il nécessite moins de drones

Scénario 3 : 1 drone pour tout Montréal

Tout le réseau est traité par un seul drone, donnant un coût minimal mais des temps de parcours très élevés.

Les résultats obtenus sont résumés ci-dessous :

Scénario	Durée estimée	Coût total
Scénario 1	6 h	1963 $
Scénario 2	26 h	1180 $
Scénario 3	195 h	168 $

Résultats pour les déneigeuses

Nous avons testé notre modèle de planification sur les opérations de déneigement dans le quartier de Verdun. Plusieurs configurations ont été simulées en variant le type de déneigeuse utilisé (Type I ou Type II) ainsi que le nombre de véhicules déployés. Le tableau suivant résume les temps d'intervention ainsi que les coûts associés pour chaque scénario étudié :

Scénario	**Type**	**Nb déneigeuses**	**Temps total (h)**	**Coût total ($)**
1	Type I	1	21h	843.0 $
2	Type I	2	10	1328.24 $
3	Type I	3	9h	1808.74 $
4	Type II	1	10	1190.16 $
5	Type II	2	5h	1980.72 $
6	Type II	3	4h	2759.29 $
7	Type II	4	4h	3559.12 $
8	Type II	5	4h	4344.6 $

3 Limites du modèle

Notre approche comporte plusieurs limites :

- Pas de prise en compte des obstacles physiques (ponts, tunnels).
- Pas de contrainte d'autonomie batterie ou météo sur les drones et les déneigeuses.
- La solution exacte (eulerize) devient difficilement calculable sur l'ensemble de Montréal.

Des extensions possibles incluraient :

- l'ajout de contraintes opérationnelles (temps de recharge, points de départ fixes),